

Introduction to Basic Python Usage and Functions

Python Syntax

- Indentation is crucial in Python.
- Python uses statements and expressions to perform operations.

- Example:

- `x = 5`

- `if x > 2:`

- `print('x is greater than 2')`

```
x = 5
if x > 2:
    print('x is greater than 2')
```

```
x is greater than 2
```

Variables and Data Types

- Variables store data values.
- Common data types: int, float, str, bool, list, dict.
- **person** is a **dict**. Dictionary (dict) is a collection of key-value pairs, where each key is unique and maps to a value.
- **name** is a **str** (String). A string is a sequence of characters and is used to represent text in Python
- **age** is an **int** (Integer). Integers in Python are whole numbers, and they can be positive, negative, or zero.
- **height** is a **float** (Floating Point Number). Floats in Python are numbers that contain a decimal point, and they are used to represent fractional numbers.
- **is_student** is a **bool** (Boolean). Booleans in Python can only have one of two values: True or False. They are used to represent truth values and are commonly used in conditional statements

```
person = {  
    'name': 'Alice',  
    'age': 30,  
    'height': 5.6,  
    'is_student': True  
}
```

Arithmetic Operators

- Arithmetic operators are used to perform mathematical operations between two or more values.
- **+ (Addition)**: Adds two numbers together.
 - Example: $5 + 3$ results in 8.
- **- (Subtraction)**: Subtracts the right operand from the left operand.
 - Example: $10 - 4$ results in 6.
- *** (Multiplication)**: Multiplies two numbers together.
 - Example: $6 * 7$ results in 42.
- **/ (Division)**: Divides the left operand by the right operand, resulting in a float.
 - Example: $10 / 3$ results in 3.3333....
- **// (Floor Division)**: Divides the left operand by the right operand and returns the largest integer less than or equal to the result.
 - Example: $10 // 3$ results in 3.
- **% (Modulus)**: Returns the remainder when the left operand is divided by the right operand.
 - Example: $10 \% 3$ results in 1.
- **** (Exponentiation)**: Raises the left operand to the power of the right operand.
 - Example: $2 ** 3$ results in 8 (2 raised to the power of 3).

Example Arithmetic Operators

```
# Arithmetic Operations
addition = 5 + 3
subtraction = 10 - 4
multiplication = 6 * 7
division = 10 / 3
floor_division = 10 // 3
modulus = 10 % 3
exponentiation = 2 ** 3

# Print the results of arithmetic operations
print("Addition (5 + 3):", addition)           # 8
print("Subtraction (10 - 4):", subtraction)    # 6
print("Multiplication (6 * 7):", multiplication) # 42
print("Division (10 / 3):", division)          # 3.333...
print("Floor Division (10 // 3):", floor_division) # 3
print("Modulus (10 % 3):", modulus)             # 1
print("Exponentiation (2 ** 3):", exponentiation) # 8
```

Comparison Operators

Comparison operators are used to compare two values and return a boolean (True or False).

- **== (Equal to)**: Returns True if the two operands are equal.
 - Example: `5 == 5` results in True.
- **!= (Not equal to)**: Returns True if the two operands are not equal.
 - Example: `5 != 3` results in True.
- **> (Greater than)**: Returns True if the left operand is greater than the right operand.
 - Example: `7 > 5` results in True.
- **< (Less than)**: Returns True if the left operand is less than the right operand.
 - Example: `3 < 8` results in True.
- **>= (Greater than or equal to)**: Returns True if the left operand is greater than or equal to the right operand.
 - Example: `5 >= 5` results in True.
- **<= (Less than or equal to)**: Returns True if the left operand is less than or equal to the right operand.
 - Example: `4 <= 6` results in True.

Comparison Operators

```
# Comparison Operations
is_equal = 5 == 5
is_not_equal = 5 != 3
is_greater = 7 > 5
is_less = 3 < 8
is_greater_equal = 5 >= 5
is_less_equal = 4 <= 6

# Print the results of comparison operations
print("Is Equal (5 == 5):", is_equal)           # True
print("Is Not Equal (5 != 3):", is_not_equal)   # True
print("Is Greater (7 > 5):", is_greater)        # True
print("Is Less (3 < 8):", is_less)              # True
print("Is Greater or Equal (5 >= 5):", is_greater_equal) # True
print("Is Less or Equal (4 <= 6):", is_less_equal) # True
```

Logical Operators

Logical operators are used to combine conditional statements.

and: Returns True if both statements are True.

- Example: True and False results in False.

or: Returns True if at least one of the statements is True.

- Example: True or False results in True.

not: Reverses the boolean value.

- Example: not True results in False.

```
# Logical Operations
and_operation = True and False
or_operation = True or False
not_operation = not True

# Print the results of logical operations
print("AND Operation (True and False):", and_operation)    # False
print("OR Operation (True or False):", or_operation)       # True
print("NOT Operation (not True):", not_operation)           # False
```


Input and Output

For interacting with users and displaying information.

- Use `input()` to get user input.
- Use `print()` to display output.

```
# Asking the user for their name
name = input("What is your name? ")

# Displaying the input received
print("Hello, " + name + "!")
```

```
What is your name? Mireilla
Hello, Mireilla!
```

The **`input("What is your name? ")`** prompts the user with the text "What is your name? ". The user's input is stored as a string in the variable `name`.

The **`print()`** function is then used to display a greeting message that includes the user's name.

Even if the user types a number, the input is still treated as a string. To perform arithmetic operations, you must convert it to an integer **`int()`** or a float.

```
: # Asking the user for their age and converting it to an integer
age = int(input("How old are you? "))

# Calculating the year the user was born
year_of_birth = 2024 - age

# Displaying the year of birth
print("You were born in " + str(year_of_birth) + ".")
```

```
How old are you? 15
You were born in 2009.
```

The result is converted back to a string using **`str()`** before being concatenated and displayed with **`print()`**.

Input and Output

```
# Asking for two numbers in one line
num1, num2 = input("Enter two numbers separated by a space: ").split()

# Converting the input to integers
num1 = int(num1)
num2 = int(num2)

# Displaying the sum
print(f"The sum of {num1} and {num2} is {num1 + num2}.")
```

```
Enter two numbers separated by a space: 23 45
The sum of 23 and 45 is 68.
```

input().split() allows the user to enter multiple values separated by a space. It splits the input string into a list of substrings. These values are then converted to integers and summed.

```
# Printing items with a custom separator
print("apple", "banana", "cherry", sep=", ")

# Printing without a newline at the end
print("Hello", end=" ")
print("World!")
```

```
apple, banana, cherry
Hello World!
```

sep: This parameter changes the separator between items in a single **print()** call. By default, it is a space.

end: This parameter changes what is printed at the end of the **print()** call. By default, it is a newline, but you can change it to a space or anything else.

Comments and Documentation

- **Single-line** comments start with **#**.
- Multi-line comments use **'''** or **"""**.

```
# This is a single-line comment
```

```
'''
```

```
|This is a multi-line comment  
that spans multiple lines.
```

```
'''
```

Defining and Calling Functions

- A function in Python is a block of reusable code that performs a specific task.
- Functions help in organising code, making it more modular, and reducing repetition.
- They can take inputs, process them, and return a result

def: This keyword is used to start the function definition.

function_name: The name you give to your function. It should be descriptive of what the function does.

parameters: Inputs to the function (optional). You can have zero, one, or more parameters.

statements: The block of code that runs when the function is called. This is indented to signify that it's part of the function.

The diagram shows a code snippet with several annotations:

- function definition:** A red line points to the `def` keyword.
- function name:** A blue line points to the word `greet`.
- parameter (this is optional):** A purple line points to the `(name):` part of the function signature.
- statement:** A purple line points to the indented `print(f'Hello, {name}!')` line.
- function call (so that it can do what is in the statement):** A black line points to the `greet('Alice')` and `greet('Bob')` lines.

```
def greet(name):  
    print(f'Hello, {name}!')
```

```
greet('Alice')  
greet('Bob')
```

Functions with Multiple Parameters /Return values

Function Definition:

- **def add(a, b):** defines a function that takes two parameters, **a** and **b**.
- Inside the function, the **result** is calculated by adding **a** and **b**.
- The function then prints the result using **print()**.

Function Calls:

- **add(5, 3)** calls the function with **5** and **3** as arguments, resulting in the output The sum of **5** and **3** is **8**.
- **add(10, 20)** calls the function with **10** and **20** as arguments, resulting in the output **The sum of 10 and 20 is 30**.

```
def add(a, b):  
    result = a + b  
    print(f'The sum of {a} and {b} is {result}')
```



```
add(5, 3)  
add(10, 20)
```

The sum of 5 and 3 is 8

The sum of 10 and 20 is 30

Function Definition:

- **def multiply(a, b):** defines a function that takes two parameters, a and b.
- The function returns the product of a and b using the **return** statement.

Function Call and Return Value:

multiply(4, 5) calls the function with 4 and 5 as arguments, and the function returns 20.

The returned value (**20**) is stored in the variable result.

The **print()** function then outputs **The product of 4 and 5 is 20**.

```
def multiply(a, b):  
    return a * b
```



```
result = multiply(4, 5)  
print(f'The product of 4 and 5 is {result}')
```

The product of 4 and 5 is 20

Functions with Default Parameters

You can define functions with default parameter values. If no argument is provided for that parameter when the function is called, the default value is used.

Function Definition:

- **def greet(name='Guest'):** defines a function with a default parameter **name** set to **'Guest'**.
- If the function is called without providing an argument, **name** will default to **'Guest'**.

```
def greet(name='Guest'):
    print(f'Hello, {name}!')
```

```
greet()
greet('Charlie')
```

```
Hello, Guest!
Hello, Charlie!
```

Function Calls:

greet() calls the function without any arguments, so it uses the default value and prints **Hello, Guest!**.

greet('Charlie') calls the function with the argument **'Charlie'**, so it prints **Hello, Charlie!**.

Scope of Variables

- Variables defined inside a function (**local variables**) are only accessible within that function.
- Variables defined outside any function (**global variables**) are accessible throughout the code.

```
x = 10 # Global variable

def foo():
    x = 5 # Local variable
    print(f'Inside function, x = {x}')

foo()
print(f'Outside function, x = {x}')
```

Inside function, x = 5
Outside function, x = 10

Global Variable:

- `x = 10` defines a global variable `x`.

Local Variable:

- Inside the function **foo()**, **x = 5** defines a local variable **x** that shadows the global **x** within the function scope.
- Inside the function, **x** is **5**, but outside the function, **x** remains **10**.

- The term "**shadowing**" in programming refers to the situation where a local variable in a function has the same name as a global variable.
- When this happens, the local variable "shadows" or hides the global variable within the scope of the function.
- This means that inside the function, any reference to that variable name will refer to the local variable, not the global one.